

COMP111: Artificial Intelligence

Intelligent Agents

Frank Wolter

Content

- ▶ The notion of intelligent agents (following Russell and Norvig);

Content

- ▶ The notion of intelligent agents (following Russell and Norvig);
- ▶ Agent function vs agent program; what **rationality** actually means;

Content

- ▶ The notion of intelligent agents (following Russell and Norvig);
- ▶ Agent function vs agent program; what **rationality** actually means;
- ▶ Task Environments (PEAS): Performance measure, environment, actuators, sensors;

Content

- ▶ The notion of intelligent agents (following Russell and Norvig);
- ▶ Agent function vs agent program; what **rationality** actually means;
- ▶ Task Environments (PEAS): Performance measure, environment, actuators, sensors;
- ▶ Task environment classification: fully observable vs partially observable, deterministic vs stochastic, episodic vs sequential, static vs dynamic, discrete vs continuous, single- vs multi-agent, known vs unknown;

Content

- ▶ The notion of intelligent agents (following Russell and Norvig);
- ▶ Agent function vs agent program; what **rationality** actually means;
- ▶ Task Environments (PEAS): Performance measure, environment, actuators, sensors;
- ▶ Task environment classification: fully observable vs partially observable, deterministic vs stochastic, episodic vs sequential, static vs dynamic, discrete vs continuous, single- vs multi-agent, known vs unknown;
- ▶ Agent classification: simple reflex agents, model-based reflex agents, goal-based agents, utility-based agents, learning agents.

Content

- ▶ The notion of intelligent agents (following Russell and Norvig);
- ▶ Agent function vs agent program; what **rationality** actually means;
- ▶ Task Environments (PEAS): Performance measure, environment, actuators, sensors;
- ▶ Task environment classification: fully observable vs partially observable, deterministic vs stochastic, episodic vs sequential, static vs dynamic, discrete vs continuous, single- vs multi-agent, known vs unknown;
- ▶ Agent classification: simple reflex agents, model-based reflex agents, goal-based agents, utility-based agents, learning agents.

Learning outcomes

Give a PEAS description of a task; classify a task environment along each dimension and say why the classification matters; identify which of the five agent architectures a system uses, and give the limitation that forces the move to the next one.

Intelligent Agents

- ▶ The intelligent entities that we engineer in AI are known as **agents**.

Intelligent Agents

- ▶ The intelligent entities that we engineer in AI are known as **agents**.
- ▶ A popular characterisation by Russell and Norvig is:
*An agent is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.*

Intelligent Agents

- ▶ The intelligent entities that we engineer in AI are known as **agents**.
- ▶ A popular characterisation by Russell and Norvig is:
*An agent is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.*
- ▶ A **human agent** has eyes, ears and other organs as sensors and hands, legs and other body parts as actuators.

Intelligent Agents

- ▶ The intelligent entities that we engineer in AI are known as **agents**.
- ▶ A popular characterisation by Russell and Norvig is:
*An agent is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.*
- ▶ A **human agent** has eyes, ears and other organs as sensors and hands, legs and other body parts as actuators.
- ▶ A **robotic agent** might have cameras as sensors and various motors as actuators.

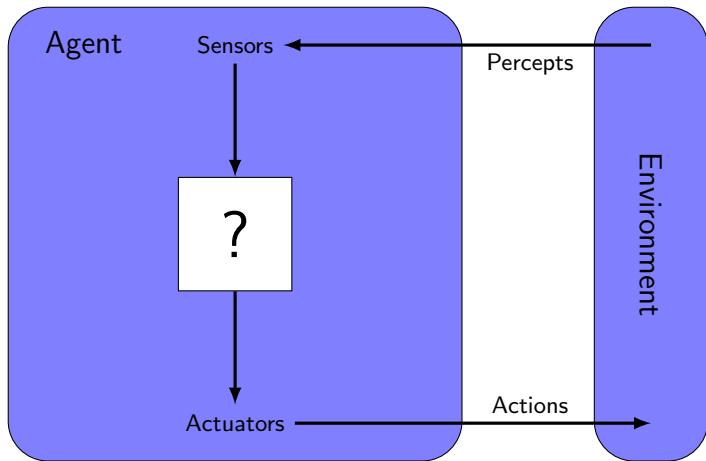
Intelligent Agents

- ▶ The intelligent entities that we engineer in AI are known as **agents**.
- ▶ A popular characterisation by Russell and Norvig is:
*An agent is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.*
- ▶ A **human agent** has eyes, ears and other organs as sensors and hands, legs and other body parts as actuators.
- ▶ A **robotic agent** might have cameras as sensors and various motors as actuators.
- ▶ A **software agent** might receive keystrokes and file contents as sensor inputs and might act on the environment by writing files. (A modern coding assistant reads a repository and edits it: same picture.)

Intelligent Agents

- ▶ The intelligent entities that we engineer in AI are known as **agents**.
- ▶ A popular characterisation by Russell and Norvig is:
*An agent is anything that can be viewed as perceiving its **environment** through **sensors** and acting upon that environment through **actuators**.*
- ▶ A **human agent** has eyes, ears and other organs as sensors and hands, legs and other body parts as actuators.
- ▶ A **robotic agent** might have cameras as sensors and various motors as actuators.
- ▶ A **software agent** might receive keystrokes and file contents as sensor inputs and might act on the environment by writing files. (A modern coding assistant reads a repository and edits it: same picture.)
- ▶ It is reasonable to say that this module is about tools for designing and implementing rational agents.

Agent



Agent function vs agent program

- ▶ A **percept** is the agent's input at one instant; the **percept sequence** is the complete history of everything the agent has ever perceived.

Agent function vs agent program

- ▶ A **percept** is the agent's input at one instant; the **percept sequence** is the complete history of everything the agent has ever perceived.
- ▶ Mathematically, an agent is described by its **agent function**

$$f : \mathcal{P}^* \longrightarrow \mathcal{A}$$

mapping every possible percept sequence to an action. *This is a specification: it says **what** the agent does.*

Agent function vs agent program

- ▶ A **percept** is the agent's input at one instant; the **percept sequence** is the complete history of everything the agent has ever perceived.
- ▶ Mathematically, an agent is described by its **agent function**

$$f : \mathcal{P}^* \longrightarrow \mathcal{A}$$

mapping every possible percept sequence to an action. *This is a specification: it says **what** the agent does.*

- ▶ The **agent program** is the concrete implementation. *It says **how**.*

Agent function vs agent program

- ▶ A **percept** is the agent's input at one instant; the **percept sequence** is the complete history of everything the agent has ever perceived.
- ▶ Mathematically, an agent is described by its **agent function**

$$f : \mathcal{P}^* \longrightarrow \mathcal{A}$$

mapping every possible percept sequence to an action. *This is a specification: it says **what** the agent does.*

- ▶ The **agent program** is the concrete implementation. *It says **how**.*
- ▶ Why not simply tabulate f ? Because the table is astronomically large: for chess it has more entries than there are atoms in the universe. **The whole art of AI is producing rational behaviour from a small program.**

Example: the vacuum world

The **vacuum world** (Russell & Norvig) is our simplest test-bed. We use it because it is small enough to write the **entire agent function** as a table — impossible for chess or driving, where that table would not fit in the universe.

Two squares, A , B ; each clean or dirty. The agent sits in one square and perceives **only that square**:

percept = (location, status).

Actions: *Left*, *Right*, *Suck*, *NoOp*.

One agent function (*suck if dirty, else move*):

Percept sequence	Action
[A, Dirty]	Suck
[A, Clean]	Right
[B, Dirty]	Suck
[B, Clean]	Left
[A, Clean]	Right
⋮	⋮

Example: the vacuum world

The **vacuum world** (Russell & Norvig) is our simplest test-bed. We use it because it is small enough to write the **entire agent function** as a table — impossible for chess or driving, where that table would not fit in the universe.

Two squares, *A*, *B*; each clean or dirty. The agent sits in one square and perceives **only that square**:

percept = (location, status).

Actions: *Left*, *Right*, *Suck*, *NoOp*.

One agent function (*suck if dirty, else move*):

Percept sequence	Action
[A, Dirty]	Suck
[A, Clean]	Right
[B, Dirty]	Suck
[B, Clean]	Left
[A, Clean]	Right
⋮	⋮

Sensible vs. rational?

The table looks *sensible* — but “rational” is defined only *relative to a performance measure*. For instance: Score +1 per clean square each step **and -1 per move**: once both squares are clean the table still says *Right*, so the agent paces forever, losing a point each step.

Rationality

Definition (Russell & Norvig)

For each possible percept sequence, a **rational agent** should select an action that is expected to **maximise its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationality

Definition (Russell & Norvig)

For each possible percept sequence, a **rational agent** should select an action that is expected to **maximise its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationality depends on four things: the performance measure, the agent's prior knowledge, the available actions, and the percept sequence to date.

Rationality

Definition (Russell & Norvig)

For each possible percept sequence, a **rational agent** should select an action that is expected to **maximise its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationality depends on four things: the performance measure, the agent's prior knowledge, the available actions, and the percept sequence to date.

- ▶ Rational $\neq$ **omniscient**: you cannot know the actual outcome, only the expected one. Crossing a road safely and being hit by a falling cargo door does not make you irrational.

Rationality

Definition (Russell & Norvig)

For each possible percept sequence, a **rational agent** should select an action that is expected to **maximise its performance measure**, given the evidence provided by the percept sequence and whatever built-in knowledge the agent has.

Rationality depends on four things: the performance measure, the agent's prior knowledge, the available actions, and the percept sequence to date.

- ▶ Rational $\neq$ **omniscient**: you cannot know the actual outcome, only the expected one. Crossing a road safely and being hit by a falling cargo door does not make you irrational.
- ▶ Rational $\neq$ **perfect**: rationality maximises *expected* performance.

Be careful what you measure

The performance measure is chosen by *us*. Getting it wrong produces perfectly rational — and useless (or dangerous) — agents.

Be careful what you measure

The performance measure is chosen by *us*. Getting it wrong produces perfectly rational — and useless (or dangerous) — agents.

- ▶ Vacuum agent rewarded for *amount of dirt collected*: it sucks up dirt, dumps it back on the floor, and sucks it up again. Forever.

Be careful what you measure

The performance measure is chosen by *us*. Getting it wrong produces perfectly rational — and useless (or dangerous) — agents.

- ▶ Vacuum agent rewarded for *amount of dirt collected*: it sucks up dirt, dumps it back on the floor, and sucks it up again. Forever.
- ▶ A boat-racing agent rewarded for *picking up bonus points* learns to drive in circles through a lagoon, never finishing the race.

Be careful what you measure

The performance measure is chosen by *us*. Getting it wrong produces perfectly rational — and useless (or dangerous) — agents.

- ▶ Vacuum agent rewarded for *amount of dirt collected*: it sucks up dirt, dumps it back on the floor, and sucks it up again. Forever.
- ▶ A boat-racing agent rewarded for *picking up bonus points* learns to drive in circles through a lagoon, never finishing the race.
- ▶ Train a recommender system (choosing the content shown next) to maximize *engagement* and it will discover that outrage maximizes engagement.

Be careful what you measure

The performance measure is chosen by *us*. Getting it wrong produces perfectly rational — and useless (or dangerous) — agents.

- ▶ Vacuum agent rewarded for *amount of dirt collected*: it sucks up dirt, dumps it back on the floor, and sucks it up again. Forever.
- ▶ A boat-racing agent rewarded for *picking up bonus points* learns to drive in circles through a lagoon, never finishing the race.
- ▶ Train a recommender system (choosing the content shown next) to maximize *engagement* and it will discover that outrage maximizes engagement.

This is called **specification gaming** (or reward hacking): the agent optimises what you *said*, not what you *meant*.

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

When we design agents to solve particular problems, we must specify the task environment as fully as possible. Four elements (PEAS) to take into account (from Russell and Norvig):

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

When we design agents to solve particular problems, we must specify the task environment as fully as possible. Four elements (PEAS) to take into account (from Russell and Norvig):

- ▶ **Performance measure**: the criteria by which we can measure the success of an agent's behaviour.

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

When we design agents to solve particular problems, we must specify the task environment as fully as possible. Four elements (PEAS) to take into account (from Russell and Norvig):

- ▶ **Performance measure**: the criteria by which we can measure the success of an agent's behaviour.
- ▶ **Environment**: the external environment that the agent inhabits.

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

When we design agents to solve particular problems, we must specify the task environment as fully as possible. Four elements (PEAS) to take into account (from Russell and Norvig):

- ▶ **Performance measure**: the criteria by which we can measure the success of an agent's behaviour.
- ▶ **Environment**: the external environment that the agent inhabits.
- ▶ **Actuators**: the means by which the agent acts within its environment.

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

When we design agents to solve particular problems, we must specify the task environment as fully as possible. Four elements (PEAS) to take into account (from Russell and Norvig):

- ▶ **Performance measure**: the criteria by which we can measure the success of an agent's behaviour.
- ▶ **Environment**: the external environment that the agent inhabits.
- ▶ **Actuators**: the means by which the agent acts within its environment.
- ▶ **Sensors**: the means by which the agent senses its environment.

Task Environments

Agents are situated within task environments, which differ in accordance with the particular problem area that the agent is designed to address.

When we design agents to solve particular problems, we must specify the task environment as fully as possible. Four elements (PEAS) to take into account (from Russell and Norvig):

- ▶ **Performance measure**: the criteria by which we can measure the success of an agent's behaviour.
- ▶ **Environment**: the external environment that the agent inhabits.
- ▶ **Actuators**: the means by which the agent acts within its environment.
- ▶ **Sensors**: the means by which the agent senses its environment.

PEAS comes *first*: before choosing an algorithm, write down what success means and what the agent can see and do.

Example 1: taxi driver

Consider a taxi driver agent. Its PEAS description might be as follows:

Example 1: taxi driver

Consider a taxi driver agent. Its PEAS description might be as follows:

- ▶ Performance measure: safe, fast, legal, comfortable, profitable, etc.;

Example 1: taxi driver

Consider a taxi driver agent. Its PEAS description might be as follows:

- ▶ Performance measure: safe, fast, legal, comfortable, profitable, etc.;
- ▶ Environment: roads, other traffic, pedestrians, customers;

Example 1: taxi driver

Consider a taxi driver agent. Its PEAS description might be as follows:

- ▶ Performance measure: safe, fast, legal, comfortable, profitable, etc.;
- ▶ Environment: roads, other traffic, pedestrians, customers;
- ▶ Actuators: steering, accelerator, brake, signal, display, etc.;

Example 1: taxi driver

Consider a taxi driver agent. Its PEAS description might be as follows:

- ▶ Performance measure: safe, fast, legal, comfortable, profitable, etc.;
- ▶ Environment: roads, other traffic, pedestrians, customers;
- ▶ Actuators: steering, accelerator, brake, signal, display, etc.;
- ▶ Sensors: cameras, sonar, speedometer, GPS, engine sensors, keyboard.

Example 1: taxi driver

Consider a taxi driver agent. Its PEAS description might be as follows:

- ▶ Performance measure: safe, fast, legal, comfortable, profitable, etc.;
- ▶ Environment: roads, other traffic, pedestrians, customers;
- ▶ Actuators: steering, accelerator, brake, signal, display, etc.;
- ▶ Sensors: cameras, sonar, speedometer, GPS, engine sensors, keyboard.

Note that the performance measure already contains a **trade-off**: fast *and* safe *and* comfortable cannot all be maximised at once. We will return to this with utility functions.

Example 2: medical diagnosis

Consider an agent used for medical diagnosis. Its PEAS description might be as follows:

Example 2: medical diagnosis

Consider an agent used for medical diagnosis. Its PEAS description might be as follows:

- ▶ Performance measure: health of patient, costs of treatment.

Example 2: medical diagnosis

Consider an agent used for medical diagnosis. Its PEAS description might be as follows:

- ▶ Performance measure: health of patient, costs of treatment.
- ▶ Environment: patient, hospital, staff.

Example 2: medical diagnosis

Consider an agent used for medical diagnosis. Its PEAS description might be as follows:

- ▶ Performance measure: health of patient, costs of treatment.
- ▶ Environment: patient, hospital, staff.
- ▶ Actuators: display questions, tests, diagnoses and treatments.

Example 2: medical diagnosis

Consider an agent used for medical diagnosis. Its PEAS description might be as follows:

- ▶ Performance measure: health of patient, costs of treatment.
- ▶ Environment: patient, hospital, staff.
- ▶ Actuators: display questions, tests, diagnoses and treatments.
- ▶ Sensors: keyboard entry of patient's symptoms, responses to questions and findings.

Example 2: medical diagnosis

Consider an agent used for medical diagnosis. Its PEAS description might be as follows:

- ▶ Performance measure: health of patient, costs of treatment.
- ▶ Environment: patient, hospital, staff.
- ▶ Actuators: display questions, tests, diagnoses and treatments.
- ▶ Sensors: keyboard entry of patient's symptoms, responses to questions and findings.

A subtlety: the agent's *actions change the percepts it will get* (ordering a test produces new evidence). Deciding *which test to order* is itself a rational decision — information has value, and a cost.

Task Environments: properties

The properties of the task environment may differ greatly, depending upon the application area. Russell and Norvig classify environments along these dimensions:

- ▶ Fully observable vs partially observable
- ▶ Deterministic vs stochastic
- ▶ Episodic vs sequential
- ▶ Static vs dynamic
- ▶ Discrete vs continuous
- ▶ Single-agent vs multi-agent
- ▶ Known vs unknown

Task Environments: properties

The properties of the task environment may differ greatly, depending upon the application area. Russell and Norvig classify environments along these dimensions:

- ▶ Fully observable vs partially observable
- ▶ Deterministic vs stochastic
- ▶ Episodic vs sequential
- ▶ Static vs dynamic
- ▶ Discrete vs continuous
- ▶ Single-agent vs multi-agent
- ▶ Known vs unknown

Why bother? Because the classification **determines the agent design**: partial observability forces internal state; stochasticity forces reasoning about probabilities; sequentiality forces search and planning. The hardest case — partially observable, stochastic, sequential, dynamic, continuous, multi-agent, unknown — is called *driving a taxi*.

Fully observable vs partially observable

Fully observable environment: one in which the agent can obtain complete, up-to-date information about the environment's state. Many moderately complex environments are only partially observable.

- ▶ Fully observable environments are more convenient: the agent does not need to maintain any internal state (including memory) to keep track of the environment.

Fully observable vs partially observable

Fully observable environment: one in which the agent can obtain complete, up-to-date information about the environment's state. Many moderately complex environments are only partially observable.

- ▶ Fully observable environments are more convenient: the agent does not need to maintain any internal state (including memory) to keep track of the environment.
- ▶ Fully observable examples: a crossword puzzle, the game of chess.

Fully observable vs partially observable

Fully observable environment: one in which the agent can obtain complete, up-to-date information about the environment's state. Many moderately complex environments are only partially observable.

- ▶ Fully observable environments are more convenient: the agent does not need to maintain any internal state (including memory) to keep track of the environment.
- ▶ Fully observable examples: a crossword puzzle, the game of chess.
- ▶ Partially observable examples: the everyday physical world (driving a car, playing football), the card game poker.

Fully observable vs partially observable

Fully observable environment: one in which the agent can obtain complete, up-to-date information about the environment's state. Many moderately complex environments are only partially observable.

- ▶ Fully observable environments are more convenient: the agent does not need to maintain any internal state (including memory) to keep track of the environment.
- ▶ Fully observable examples: a crossword puzzle, the game of chess.
- ▶ Partially observable examples: the everyday physical world (driving a car, playing football), the card game poker.
- ▶ Two causes of partial observability: **noisy or inaccurate sensors**, and **missing sensors** (a vacuum agent with no sensor for the other room). The extreme case, no sensors at all, is called an *unobservable* environment — surprisingly, a rational agent can still sometimes guarantee success.

Deterministic vs stochastic

Deterministic environment: one in which any action has a single guaranteed effect — there is no uncertainty about the state that will result from performing an action. In a stochastic environment the effect is not guaranteed.

This definition applies **from the point of view of the agent**.

- ▶ If the environment is deterministic except for the actions of other agents, it is said to be *strategic*.

Deterministic vs stochastic

Deterministic environment: one in which any action has a single guaranteed effect — there is no uncertainty about the state that will result from performing an action. In a stochastic environment the effect is not guaranteed.

This definition applies **from the point of view of the agent**.

- ▶ If the environment is deterministic except for the actions of other agents, it is said to be *strategic*.
- ▶ Deterministic examples: a crossword puzzle, the game of chess.

Deterministic vs stochastic

Deterministic environment: one in which any action has a single guaranteed effect — there is no uncertainty about the state that will result from performing an action. In a stochastic environment the effect is not guaranteed.

This definition applies **from the point of view of the agent**.

- ▶ If the environment is deterministic except for the actions of other agents, it is said to be *strategic*.
- ▶ Deterministic examples: a crossword puzzle, the game of chess.
- ▶ Stochastic examples: medical diagnosis, the card game poker, driving a car (the physical world).

Deterministic vs stochastic

Deterministic environment: one in which any action has a single guaranteed effect — there is no uncertainty about the state that will result from performing an action. In a stochastic environment the effect is not guaranteed.

This definition applies **from the point of view of the agent**.

- ▶ If the environment is deterministic except for the actions of other agents, it is said to be *strategic*.
- ▶ Deterministic examples: a crossword puzzle, the game of chess.
- ▶ Stochastic examples: medical diagnosis, the card game poker, driving a car (the physical world).
- ▶ Careful: the real world is *arguably* deterministic — but it is so complex, and our sensors so poor, that we *must* treat it as stochastic. A taxi's wheels really might slip. Uncertainty is a property of the agent's knowledge, not only of the world.

Episodic vs sequential

Episodic environment: one where the performance of the agent depends on a number of discrete episodes with no link between its performance in different episodes. In sequential environments different episodes are linked.

- ▶ In episodic environments the agent can decide what action to perform based only on the current episode, without reasoning about interactions between this and future episodes.

Episodic vs sequential

Episodic environment: one where the performance of the agent depends on a number of discrete episodes with no link between its performance in different episodes. In sequential environments different episodes are linked.

- ▶ In episodic environments the agent can decide what action to perform based only on the current episode, without reasoning about interactions between this and future episodes.
- ▶ In sequential environments the current decision could affect all future decisions.

Episodic vs sequential

Episodic environment: one where the performance of the agent depends on a number of discrete episodes with no link between its performance in different episodes. In sequential environments different episodes are linked.

- ▶ In episodic environments the agent can decide what action to perform based only on the current episode, without reasoning about interactions between this and future episodes.
- ▶ In sequential environments the current decision could affect all future decisions.
- ▶ Episodic examples: a mail sorting system, defect detection on an assembly line, classifying an image.

Episodic vs sequential

Episodic environment: one where the performance of the agent depends on a number of discrete episodes with no link between its performance in different episodes. In sequential environments different episodes are linked.

- ▶ In episodic environments the agent can decide what action to perform based only on the current episode, without reasoning about interactions between this and future episodes.
- ▶ In sequential environments the current decision could affect all future decisions.
- ▶ Episodic examples: a mail sorting system, defect detection on an assembly line, classifying an image.
- ▶ Sequential examples: a crossword puzzle, the card game poker, driving a car, chess.

Episodic vs sequential

Episodic environment: one where the performance of the agent depends on a number of discrete episodes with no link between its performance in different episodes. In sequential environments different episodes are linked.

- ▶ In episodic environments the agent can decide what action to perform based only on the current episode, without reasoning about interactions between this and future episodes.
- ▶ In sequential environments the current decision could affect all future decisions.
- ▶ Episodic examples: a mail sorting system, defect detection on an assembly line, classifying an image.
- ▶ Sequential examples: a crossword puzzle, the card game poker, driving a car, chess.
- ▶ Note: most machine learning as you will meet it (classification) is *episodic*; most *agent* problems are *sequential*. Sequential is much harder: a good move now may lose the game in twenty moves.

Static vs dynamic

Static environment: one that can be assumed to remain unchanged while the agent is deliberating.

Dynamic environment: one that has other processes operating on it, and hence changes while the agent is deliberating.

Static environments are easier: the agent need not keep observing the environment while deciding how to act, nor worry about time elapsing.

- ▶ Static examples: a crossword puzzle, the card game poker, the game of backgammon.

Static vs dynamic

Static environment: one that can be assumed to remain unchanged while the agent is deliberating.

Dynamic environment: one that has other processes operating on it, and hence changes while the agent is deliberating.

Static environments are easier: the agent need not keep observing the environment while deciding how to act, nor worry about time elapsing.

- ▶ Static examples: a crossword puzzle, the card game poker, the game of backgammon.
- ▶ Dynamic examples: medical diagnosis, the physical world, driving a car.

Static vs dynamic

Static environment: one that can be assumed to remain unchanged while the agent is deliberating.

Dynamic environment: one that has other processes operating on it, and hence changes while the agent is deliberating.

Static environments are easier: the agent need not keep observing the environment while deciding how to act, nor worry about time elapsing.

- ▶ Static examples: a crossword puzzle, the card game poker, the game of backgammon.
- ▶ Dynamic examples: medical diagnosis, the physical world, driving a car.
- ▶ *Semi-dynamic*: the environment does not change, but the agent's *score* does — e.g. chess with a clock. Thinking is never free.

Discrete vs continuous

Discrete environment: one that contains a fixed, finite number of distinct states.

The distinction applies to the state of the environment, the way in which time is handled, and the percepts and actions of the agent.

- ▶ Discrete examples: a crossword puzzle, the game of chess (finitely many positions and moves).

Discrete vs continuous

Discrete environment: one that contains a fixed, finite number of distinct states.

The distinction applies to the state of the environment, the way in which time is handled, and the percepts and actions of the agent.

- ▶ Discrete examples: a crossword puzzle, the game of chess (finitely many positions and moves).
- ▶ Continuous examples: driving a car (steering angle, speed and camera input all vary continuously); medical diagnosis (dosages).

Discrete vs continuous

Discrete environment: one that contains a fixed, finite number of distinct states.

The distinction applies to the state of the environment, the way in which time is handled, and the percepts and actions of the agent.

- ▶ Discrete examples: a crossword puzzle, the game of chess (finitely many positions and moves).
- ▶ Continuous examples: driving a car (steering angle, speed and camera input all vary continuously); medical diagnosis (dosages).
- ▶ Continuous environments provide greater challenges for agent designers: you cannot enumerate the states, so you need approximation — which is exactly what neural networks provide.

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour?*

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour*?
 - ▶ *Competitive:* chess, poker — my gain is your loss.

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour?*
 - ▶ *Competitive:* chess, poker — my gain is your loss.
 - ▶ *Cooperative:* two self-driving cars avoiding a collision; RoboCup team-mates.

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour*?
 - ▶ *Competitive*: chess, poker — my gain is your loss.
 - ▶ *Cooperative*: two self-driving cars avoiding a collision; RoboCup team-mates.
 - ▶ Multi-agent environments introduce communication, negotiation and randomised behaviour (being predictable is a weakness).

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour*?
 - ▶ *Competitive*: chess, poker — my gain is your loss.
 - ▶ *Cooperative*: two self-driving cars avoiding a collision; RoboCup team-mates.
 - ▶ Multi-agent environments introduce communication, negotiation and randomised behaviour (being predictable is a weakness).
- ▶ **Known vs unknown:** this is about the *agent's knowledge of the rules of the environment*, not about observability.

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour*?
 - ▶ *Competitive*: chess, poker — my gain is your loss.
 - ▶ *Cooperative*: two self-driving cars avoiding a collision; RoboCup team-mates.
 - ▶ Multi-agent environments introduce communication, negotiation and randomised behaviour (being predictable is a weakness).
- ▶ **Known vs unknown:** this is about the *agent's knowledge of the rules of the environment*, not about observability.
 - ▶ Known but partially observable: solitaire — you know the rules, but not the face-down cards.

Two more dimensions

- ▶ **Single-agent vs multi-agent:** is any other entity in the environment best modelled as *maximising a performance measure that depends on my behaviour*?
 - ▶ *Competitive*: chess, poker — my gain is your loss.
 - ▶ *Cooperative*: two self-driving cars avoiding a collision; RoboCup team-mates.
 - ▶ Multi-agent environments introduce communication, negotiation and randomised behaviour (being predictable is a weakness).
- ▶ **Known vs unknown:** this is about the *agent's knowledge of the rules of the environment*, not about observability.
 - ▶ Known but partially observable: solitaire — you know the rules, but not the face-down cards.
 - ▶ Unknown but fully observable: a new video game — you see the whole screen, but must discover what the buttons do.

Interactive: classify these task environments

Task	Observable	Determ.	Episodic	Static	Discrete	Agents
Crossword	full	determ.	sequential	static	discrete	single
Chess + clock	full	strategic	sequential	semi	discrete	multi
Poker	partial	stochastic	sequential	static	discrete	multi
Medical diagnosis	partial	stochastic	sequential	dynamic	continuous	single
Image classification	full	determ.	episodic	static	continuous	single
Taxi driving	partial	stochastic	sequential	dynamic	continuous	multi

Interactive: classify these task environments

Task	Observable	Determ.	Episodic	Static	Discrete	Agents
Crossword	full	determ.	sequential	static	discrete	single
Chess + clock	full	strategic	sequential	semi	discrete	multi
Poker	partial	stochastic	sequential	static	discrete	multi
Medical diagnosis	partial	stochastic	sequential	dynamic	continuous	single
Image classification	full	determ.	episodic	static	continuous	single
Taxi driving	partial	stochastic	sequential	dynamic	continuous	multi

Your turn

Classify (a) a spam filter, (b) a robot vacuum in a real flat with a cat, (c) an LLM agent fixing a bug in a code repository. Which dimension is hardest to decide?

Intelligent agents: a classification

We classify intelligent agents according to how they map percepts to actions. Russell and Norvig identify five basic kinds of agents that underpin most intelligent systems:

- ▶ Simple reflex agents

Intelligent agents: a classification

We classify intelligent agents according to how they map percepts to actions. Russell and Norvig identify five basic kinds of agents that underpin most intelligent systems:

- ▶ Simple reflex agents
- ▶ Model-based reflex agents

Intelligent agents: a classification

We classify intelligent agents according to how they map percepts to actions. Russell and Norvig identify five basic kinds of agents that underpin most intelligent systems:

- ▶ Simple reflex agents
- ▶ Model-based reflex agents
- ▶ Goal-based agents

Intelligent agents: a classification

We classify intelligent agents according to how they map percepts to actions. Russell and Norvig identify five basic kinds of agents that underpin most intelligent systems:

- ▶ Simple reflex agents
- ▶ Model-based reflex agents
- ▶ Goal-based agents
- ▶ Utility-based agents

Intelligent agents: a classification

We classify intelligent agents according to how they map percepts to actions. Russell and Norvig identify five basic kinds of agents that underpin most intelligent systems:

- ▶ Simple reflex agents
- ▶ Model-based reflex agents
- ▶ Goal-based agents
- ▶ Utility-based agents
- ▶ Learning agents

Intelligent agents: a classification

We classify intelligent agents according to how they map percepts to actions. Russell and Norvig identify five basic kinds of agents that underpin most intelligent systems:

- ▶ Simple reflex agents
- ▶ Model-based reflex agents
- ▶ Goal-based agents
- ▶ Utility-based agents
- ▶ Learning agents

Read the list as a **ladder**: each step is forced by a limitation of the one below. Ask at every step: *what can this agent not do?*

Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.

Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.
- ▶ Implemented using condition–action rules such as
if the car in front is braking, then initiate braking

Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.
- ▶ Implemented using condition–action rules such as
if the car in front is braking, then initiate braking
- ▶ Simple to implement, very fast — and of very limited intelligence. They cannot

Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.
- ▶ Implemented using condition–action rules such as
if the car in front is braking, then initiate braking
- ▶ Simple to implement, very fast — and of very limited intelligence. They cannot
 - ▶ play football: to decide the next move, a striker has to remember that a second ago there was a defender four metres away;

Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.
- ▶ Implemented using condition–action rules such as
if the car in front is braking, then initiate braking
- ▶ Simple to implement, very fast — and of very limited intelligence. They cannot
 - ▶ play football: to decide the next move, a striker has to remember that a second ago there was a defender four metres away;
 - ▶ play poker: the player has to remember the history of the current game;

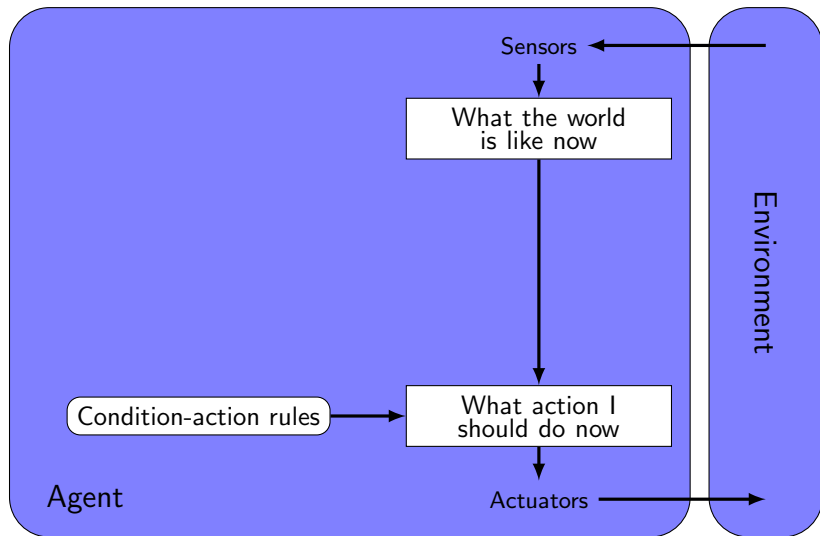
Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.
- ▶ Implemented using condition–action rules such as
if the car in front is braking, then initiate braking
- ▶ Simple to implement, very fast — and of very limited intelligence. They cannot
 - ▶ play football: to decide the next move, a striker has to remember that a second ago there was a defender four metres away;
 - ▶ play poker: the player has to remember the history of the current game;
 - ▶ drive a car: to decide whether to overtake, the driver must remember whether another car was overtaking a few seconds ago.

Simple reflex agents

- ▶ Simple reflex agents select the action to execute based only upon the **current percept**; they ignore the percept history.
- ▶ Implemented using condition–action rules such as
 - if the car in front is braking, then initiate braking*
- ▶ Simple to implement, very fast — and of very limited intelligence. They cannot
 - ▶ play football: to decide the next move, a striker has to remember that a second ago there was a defender four metres away;
 - ▶ play poker: the player has to remember the history of the current game;
 - ▶ drive a car: to decide whether to overtake, the driver must remember whether another car was overtaking a few seconds ago.
- ▶ In a partially observable environment they can also get stuck in **infinite loops** (a vacuum agent with a broken dirt sensor endlessly moving left and right). A pinch of randomness helps.

Simple Reflex Agent



Model-based reflex agents

- ▶ Model-based reflex agents maintain an **internal state** (memory) that depends upon the percept history.

Model-based reflex agents

- ▶ Model-based reflex agents maintain an **internal state** (memory) that depends upon the percept history.
- ▶ The current percept is combined with the previous internal state to update the description of the current state.

Model-based reflex agents

- ▶ Model-based reflex agents maintain an **internal state** (memory) that depends upon the percept history.
- ▶ The current percept is combined with the previous internal state to update the description of the current state.
- ▶ This is how they cope with **partial observability**: the state summarises what the agent cannot currently see.

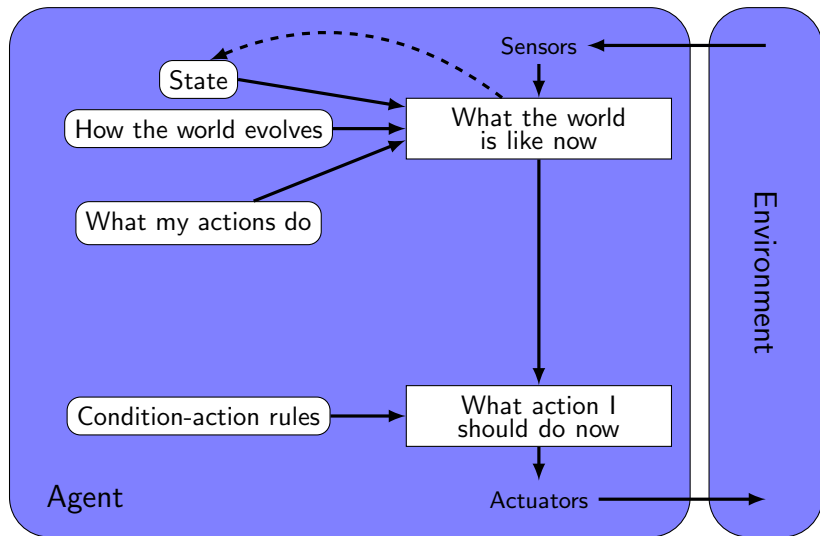
Model-based reflex agents

- ▶ Model-based reflex agents maintain an **internal state** (memory) that depends upon the percept history.
- ▶ The current percept is combined with the previous internal state to update the description of the current state.
- ▶ This is how they cope with **partial observability**: the state summarises what the agent cannot currently see.
- ▶ They need a **model** of 'how the world works':
 - ▶ how the world changes independently of the agent (the car ahead keeps moving);
 - ▶ how the world changes as a result of the agent's actions (if I turn the wheel, I change lane).

Model-based reflex agents

- ▶ Model-based reflex agents maintain an **internal state** (memory) that depends upon the percept history.
- ▶ The current percept is combined with the previous internal state to update the description of the current state.
- ▶ This is how they cope with **partial observability**: the state summarises what the agent cannot currently see.
- ▶ They need a **model** of 'how the world works':
 - ▶ how the world changes independently of the agent (the car ahead keeps moving);
 - ▶ how the world changes as a result of the agent's actions (if I turn the wheel, I change lane).
- ▶ Still reflex: the rules say *what to do now*, not *where the agent is trying to get to*. A model-based agent at a junction knows exactly what the world looks like — and has no idea whether to turn left or right.

Model-Based Reflex Agent



Goal-based agents

- ▶ Goal-based agents select actions in order to achieve desirable states of the environment: **goals**.

Goal-based agents

- ▶ Goal-based agents select actions in order to achieve desirable states of the environment: **goals**.
- ▶ Knowledge of the current state does not automatically tell the agent what to do: it must also know *what it wants* and *what its actions will bring about*.

Goal-based agents

- ▶ Goal-based agents select actions in order to achieve desirable states of the environment: **goals**.
- ▶ Knowledge of the current state does not automatically tell the agent what to do: it must also know *what it wants* and *what its actions will bring about*.
- ▶ Decision making becomes complicated when long sequences of actions are needed: **search** and **planning** may be required (the next sections of this module).

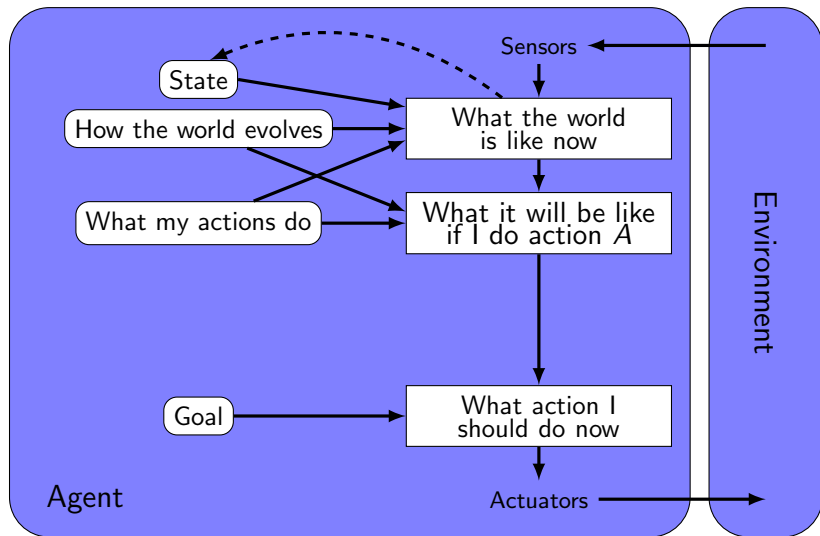
Goal-based agents

- ▶ Goal-based agents select actions in order to achieve desirable states of the environment: **goals**.
- ▶ Knowledge of the current state does not automatically tell the agent what to do: it must also know *what it wants* and *what its actions will bring about*.
- ▶ Decision making becomes complicated when long sequences of actions are needed: **search** and **planning** may be required (the next sections of this module).
- ▶ Big advantage: **flexibility**. If it starts raining, we update the model (“braking distances are longer”), and the behaviour changes automatically. A reflex agent would need all of its rules rewritten by hand.

Goal-based agents

- ▶ Goal-based agents select actions in order to achieve desirable states of the environment: **goals**.
- ▶ Knowledge of the current state does not automatically tell the agent what to do: it must also know *what it wants* and *what its actions will bring about*.
- ▶ Decision making becomes complicated when long sequences of actions are needed: **search** and **planning** may be required (the next sections of this module).
- ▶ Big advantage: **flexibility**. If it starts raining, we update the model (“braking distances are longer”), and the behaviour changes automatically. A reflex agent would need all of its rules rewritten by hand.
- ▶ Limitation: a goal is a *binary* test — achieved or not. It cannot distinguish “arrived, safely, in 20 minutes” from “arrived, terrifyingly, in 19 minutes”.

Goal-Based Agent



Utility-based agents

- ▶ Utility-based agents use a **utility function** to compare the **desirability** of the different states that result from actions.

Utility-based agents

- ▶ Utility-based agents use a **utility function** to compare the **desirability** of the different states that result from actions.
- ▶ Many action sequences may satisfy a goal — but which is the most desirable? (Many routes reach the airport; they differ in time, cost, risk and comfort.)

Utility-based agents

- ▶ Utility-based agents use a **utility function** to compare the **desirability** of the different states that result from actions.
- ▶ Many action sequences may satisfy a goal — but which is the most desirable? (Many routes reach the airport; they differ in time, cost, risk and comfort.)
- ▶ The utility function maps a state, or a sequence of states, onto a number giving its **usefulness** to the agent. The agent tries to **maximise expected utility**.

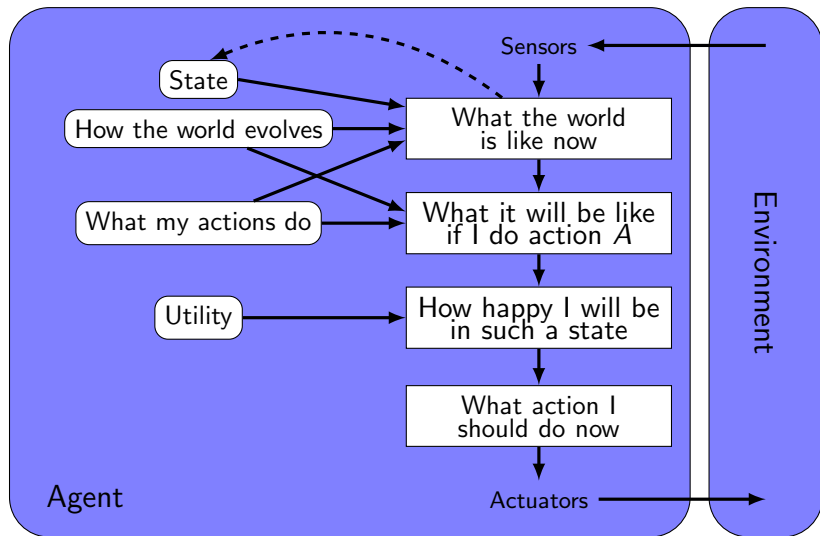
Utility-based agents

- ▶ Utility-based agents use a **utility function** to compare the **desirability** of the different states that result from actions.
- ▶ Many action sequences may satisfy a goal — but which is the most desirable? (Many routes reach the airport; they differ in time, cost, risk and comfort.)
- ▶ The utility function maps a state, or a sequence of states, onto a number giving its **usefulness** to the agent. The agent tries to **maximise expected utility**.
- ▶ This allows two things goals cannot express:
 - ▶ **trade-offs** between conflicting goals (fast vs safe);
 - ▶ rational behaviour under **uncertainty**: weigh the utility of each outcome by its probability.

Utility-based agents

- ▶ Utility-based agents use a **utility function** to compare the **desirability** of the different states that result from actions.
- ▶ Many action sequences may satisfy a goal — but which is the most desirable? (Many routes reach the airport; they differ in time, cost, risk and comfort.)
- ▶ The utility function maps a state, or a sequence of states, onto a number giving its **usefulness** to the agent. The agent tries to **maximise expected utility**.
- ▶ This allows two things goals cannot express:
 - ▶ **trade-offs** between conflicting goals (fast vs safe);
 - ▶ rational behaviour under **uncertainty**: weigh the utility of each outcome by its probability.
- ▶ The performance measure (ours, external) and the utility function (the agent's, internal) should agree — when they do not, you get specification gaming.

Utility-Based Agent



Learning agents

How does the agent achieve its goal, or maximise its utility, in an **unknown** environment? Which action to choose? Learning is the fundamental idea — and it turns *any* of the previous architectures into a learning agent.

Learning agents

How does the agent achieve its goal, or maximise its utility, in an **unknown** environment? Which action to choose? Learning is the fundamental idea — and it turns *any* of the previous architectures into a learning agent.

Four components (Russell & Norvig):

- ▶ **Performance element:** what we have called “the agent” so far — it selects actions.

Learning agents

How does the agent achieve its goal, or maximise its utility, in an **unknown** environment? Which action to choose? Learning is the fundamental idea — and it turns *any* of the previous architectures into a learning agent.

Four components (Russell & Norvig):

- ▶ **Performance element**: what we have called “the agent” so far — it selects actions.
- ▶ **Critic**: judges how the agent is doing against a fixed **performance standard** (the agent cannot mark its own homework using its own utility function).

Learning agents

How does the agent achieve its goal, or maximise its utility, in an **unknown** environment? Which action to choose? Learning is the fundamental idea — and it turns *any* of the previous architectures into a learning agent.

Four components (Russell & Norvig):

- ▶ **Performance element**: what we have called “the agent” so far — it selects actions.
- ▶ **Critic**: judges how the agent is doing against a fixed **performance standard** (the agent cannot mark its own homework using its own utility function).
- ▶ **Learning element**: uses the critic’s feedback to improve the performance element.

Learning agents

How does the agent achieve its goal, or maximise its utility, in an **unknown** environment? Which action to choose? Learning is the fundamental idea — and it turns *any* of the previous architectures into a learning agent.

Four components (Russell & Norvig):

- ▶ **Performance element**: what we have called “the agent” so far — it selects actions.
- ▶ **Critic**: judges how the agent is doing against a fixed **performance standard** (the agent cannot mark its own homework using its own utility function).
- ▶ **Learning element**: uses the critic’s feedback to improve the performance element.
- ▶ **Problem generator**: suggests exploratory actions that are suboptimal now but informative — the reason a learning taxi driver must occasionally try a new route.

Learning agents

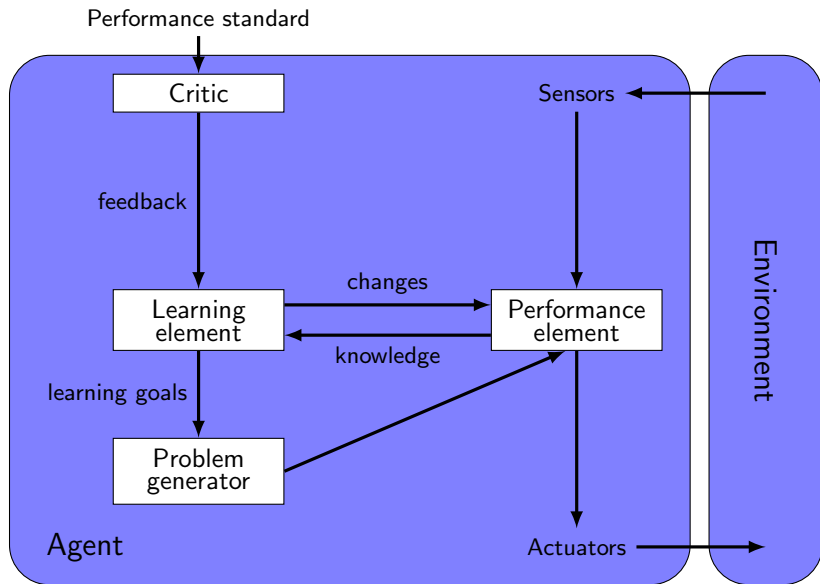
How does the agent achieve its goal, or maximise its utility, in an **unknown** environment? Which action to choose? Learning is the fundamental idea — and it turns *any* of the previous architectures into a learning agent.

Four components (Russell & Norvig):

- ▶ **Performance element**: what we have called “the agent” so far — it selects actions.
- ▶ **Critic**: judges how the agent is doing against a fixed **performance standard** (the agent cannot mark its own homework using its own utility function).
- ▶ **Learning element**: uses the critic’s feedback to improve the performance element.
- ▶ **Problem generator**: suggests exploratory actions that are suboptimal now but informative — the reason a learning taxi driver must occasionally try a new route.

This is precisely the **exploration vs exploitation** dilemma, and the basis of **reinforcement learning**.

Learning Agent



Where do today's systems sit?

- ▶ A thermostat, or a spam filter: **simple reflex**.

Where do today's systems sit?

- ▶ A thermostat, or a spam filter: **simple reflex**.
- ▶ A robot vacuum building a map of your flat: **model-based**.

Where do today's systems sit?

- ▶ A thermostat, or a spam filter: **simple reflex**.
- ▶ A robot vacuum building a map of your flat: **model-based**.
- ▶ A route planner, a chess engine: **goal-based** (with search).

Where do today's systems sit?

- ▶ A thermostat, or a spam filter: **simple reflex**.
- ▶ A robot vacuum building a map of your flat: **model-based**.
- ▶ A route planner, a chess engine: **goal-based** (with search).
- ▶ A self-driving car balancing speed, safety and comfort: **utility-based**.

Where do today's systems sit?

- ▶ A thermostat, or a spam filter: **simple reflex**.
- ▶ A robot vacuum building a map of your flat: **model-based**.
- ▶ A route planner, a chess engine: **goal-based** (with search).
- ▶ A self-driving car balancing speed, safety and comfort: **utility-based**.
- ▶ An LLM assistant:
 - ▶ *Environment* = the repository and the human;
 - ▶ *Percepts* = the user's instructions (unsolicited, arrive any time) *and* outputs (solicited — file contents, test results);
 - ▶ *Actions* = gaining knowledge (read, test) or world-changing (edit file *and* text back to the user);
 - ▶ *Internal state* = the context window — which is why it forgets;
 - ▶ *Performance measure* - very hard to write down.

Where do today's systems sit?

- ▶ A thermostat, or a spam filter: **simple reflex**.
- ▶ A robot vacuum building a map of your flat: **model-based**.
- ▶ A route planner, a chess engine: **goal-based** (with search).
- ▶ A self-driving car balancing speed, safety and comfort: **utility-based**.
- ▶ An LLM assistant:
 - ▶ *Environment* = the repository and the human;
 - ▶ *Percepts* = the user's instructions (unsolicited, arrive any time) *and* outputs (solicited — file contents, test results);
 - ▶ *Actions* = gaining knowledge (read, test) or world-changing (edit file *and* text back to the user);
 - ▶ *Internal state* = the context window — which is why it forgets;
 - ▶ *Performance measure* - very hard to write down.

The 1995 vocabulary describes 2026 systems exactly. That is why we teach it.

Summary

- ▶ An agent perceives an environment through sensors and acts through actuators; it is specified by an **agent function** and implemented by an **agent program**.

Summary

- ▶ An agent perceives an environment through sensors and acts through actuators; it is specified by an **agent function** and implemented by an **agent program**.
- ▶ A **rational** agent maximises the *expected* value of a performance measure — which we choose, and can easily choose badly.

Summary

- ▶ An agent perceives an environment through sensors and acts through actuators; it is specified by an **agent function** and implemented by an **agent program**.
- ▶ A **rational** agent maximises the *expected* value of a performance measure — which we choose, and can easily choose badly.
- ▶ **PEAS** specifies a task; the **seven dimensions** classify its environment and thereby dictate the agent design.

Summary

- ▶ An agent perceives an environment through sensors and acts through actuators; it is specified by an **agent function** and implemented by an **agent program**.
- ▶ A **rational** agent maximises the *expected* value of a performance measure — which we choose, and can easily choose badly.
- ▶ **PEAS** specifies a task; the **seven dimensions** classify its environment and thereby dictate the agent design.
- ▶ Five architectures, each fixing a limitation of the last: simple reflex → model-based → goal-based → utility-based → learning.

Summary

- ▶ An agent perceives an environment through sensors and acts through actuators; it is specified by an **agent function** and implemented by an **agent program**.
- ▶ A **rational** agent maximises the *expected* value of a performance measure — which we choose, and can easily choose badly.
- ▶ **PEAS** specifies a task; the **seven dimensions** classify its environment and thereby dictate the agent design.
- ▶ Five architectures, each fixing a limitation of the last: simple reflex → model-based → goal-based → utility-based → learning.

Reading

Russell & Norvig, *Artificial Intelligence: A Modern Approach*, Chapter 2.

Next: problem solving by search — how a goal-based agent actually finds its action sequence.